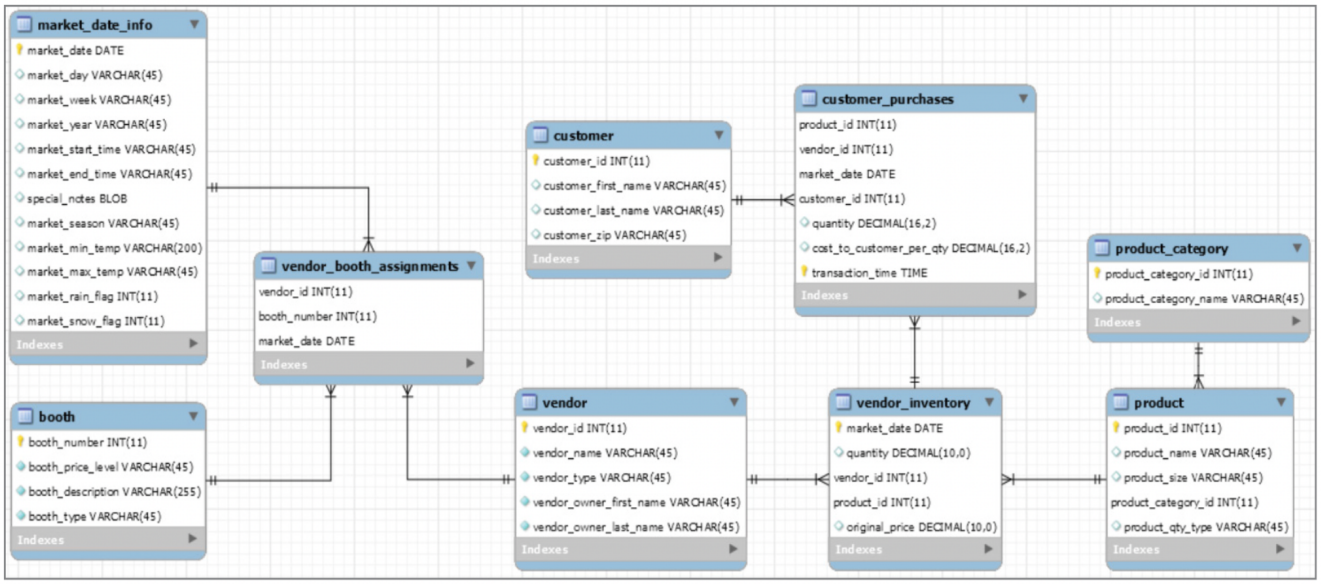
SQL - 02 - Filtering and subqueries

Problem Statement:

You are a Data Analyst at the Reliance Fresh. You have been tasked to study the Farmer's market - *Mandis*.

Dataset: Farmer's Market Database



In the last lecture, we ended at inline calculation.

Run the query and show that you **need to round off those values.**

Can we round off the price column to just 2 decimal places. - ROUND function

> Functions in SQL

- A SQL function is a piece of code that takes inputs that you give it (which are called parameters), performs some operation on those inputs, and returns a value.
- You can use **functions inline** in your query to modify the raw values from the database tables before displaying them in the output.

Function Syntax

FUNCTION_NAME([parameter 1],[parameter 2], ...[parameter n])

- Each bracketed item shown is a placeholder for an input parameter.
- Parameters go inside the parentheses following the function name and are separated by commas.
- The input parameter might be a field name or a value.

ROUND()

- In the last, the “price” field was displayed with four digits after the decimal point.
- Let’s say we wanted to display the number rounded to the nearest penny (in US dollars), which is two digits after the decimal. That can be accomplished using the ROUND() function.

```
SELECT
    market_date,
    customer_id,
    vendor_id,
    ROUND(quantity * cost_to_customer_per_qty, 2) AS price
 FROM farmers_market.customer_purchases
 LIMIT 10
```

TIP: The ROUND() function can also accept negative numbers for the second parameter, to **round digits that are to the left of the decimal point.** For example, SELECT ROUND(1245, -2) will return a value of 1200.

Other similar functions:

- **CEIL**
- **FLOOR**
- **LEAST/GREATEST - select greatest(4,5,6,7,1,2,3);**

Transition -> Up until now, we have manipulated just numbers, what about strings?

Concatenating Strings: How to work with strings in SQL?

There are also inline functions that can be used to modify string values in SQL, as well.

In our customer table, there are separate columns for each customer's first and last names. Let's quickly look at the customer table.

```
SELECT *
FROM farmers_market.customer
LIMIT 5
```

Question: We want to merge each customer's name into a single column that contains the first name, then a space, and then the last name.

- We can accomplish that by using the CONCAT() function.
- The list of string values you want to merge together are entered into the CONCAT() function as parameters.
- A space can be included by surrounding it with quotes.

```
SELECT

customer_id,

CONCAT(customer_first_name, " ", customer_last_name) AS customer_name

FROM farmers_market.customer

LIMIT 5
```

- You can now order by names, their first names and their last names:

```
SELECT
    customer_id,
    CONCAT(customer_first_name, " ", customer_last_name) AS customer_name
 FROM farmers_market.customer
 ORDER BY customer_last_name, customer_first_name
 LIMIT 5
```

- It's also possible to nest functions inside other functions, which the SQL interpreter executes **from the "inside" to the "outside."**
- UPPER() is a function that capitalizes string values.
- We can enclose the CONCAT() function inside it to uppercase the full name.
- Let's also change the order of the concatenation parameters to put the last name first, and add a comma after the last name.

```
SELECT
    customer_id,
    UPPER(CONCAT(customer_last_name, ", ", customer_first_name)) AS customer_name
 FROM farmers_market.customer
 ORDER BY customer_last_name, customer_first_name
 LIMIT 5
```

- Because the CONCAT() function is contained inside the parentheses of the UPPER() function, the concatenation is performed first, and then the combined string is uppercased.

Transition -> So, now you know how to query data in a straightforward manner, you can do some inline calculations and sort them.

But one of the most common requirements is to filter rows based on certain conditions.

Question: Extract all the product names that are part of product category 1

Filtering data - The WHERE Clause

- The WHERE clause is the part of the SELECT statement in which you list conditions that are used to determine which rows in the table should be included in the results set.
- In other words, the WHERE clause is used for filtering.

The WHERE clause goes after the FROM statement and before any GROUP BY, ORDER BY, or LIMIT statements in the SELECT query

Let's get those product names that are in category 1:

```
SELECT
product_id, product_name, product_category_id
FROM farmers_market.product
WHERE product_category_id = 1
LIMIT 5
```

Question: Print a report of everything ***customer_id 4*** has ever purchased at the farmer's market, sorted by market date, vendor ID, and product ID.

```
SELECT
        market_date, customer_id,
```

```
        vendor_id, product_id,
        quantity,
        quantity * cost_to_customer_per_qty AS price
FROM farmers_market.customer_purchases
WHERE customer_id = 4
ORDER BY market_date, vendor_id, product_id
LIMIT 5
```

Behind the scenes - How WHERE clause works?

- Each of the conditional statements (like “customer_id = 4”) listed in the WHERE clause will evaluate to TRUE or FALSE for each row, and only the rows for which the combination of conditions evaluates to TRUE will be returned.

Filtering on multiple conditions - using operators - “AND”, “OR”, “NOT”

- Conditions with OR between them will jointly evaluate to TRUE, meaning the row will be returned, if any of the clauses are TRUE.
- Conditions with AND between them will only evaluate to TRUE in combination if all of the clauses evaluate to TRUE. Otherwise, the row will not be returned.
- **Remember that NOT flips the following boolean value to its opposite (TRUE becomes FALSE, and vice versa).**

Question: Get all the product info for products with id between 3 and 8 (not inclusive) and of product with id 10.

```
SELECT
    product_id,
```

```
    product_name
 FROM farmers_market.product
 WHERE
    product_id = 10
    OR (product_id > 3
    AND product_id < 8)
```

Now, check this

```
SELECT
    product_id,
    product_name
 FROM farmers_market.product
 WHERE
    (product_id = 10
    OR product_id > 3)
    AND product_id < 8
```

Explanation:

- When the product ID is 10, the WHERE clause in the first query is evaluated as:

```
TRUE OR (TRUE AND FALSE) = TRUE OR (FALSE) = TRUE
```

- and the WHERE clause in the second query is evaluated as:

```
(TRUE OR TRUE) AND FALSE = (TRUE) AND FALSE = FALSE
```

Multi-column filtering - WHERE clauses can also impose conditions using values in multiple columns.

Question: Find the details of purchases made by customer 4 at vendor 7, we could use the following query:

```
SELECT
    market_date,
    customer_id,
    vendor_id,
    quantity * cost_to_customer_per_qty AS price
 FROM farmers_market.customer_purchases
 WHERE
    customer_id = 4
    AND vendor_id = 7
```

Filtering based on strings

Question: Find the customer detail with the first name of “Carlos” or the last name of “Diaz,”:

```
SELECT
    customer_id,
    customer_first_name,
    customer_last_name
 FROM farmers_market.customer
 WHERE
    customer_first_name = 'Carlos'
    OR customer_last_name = 'Diaz'
```

Question: If you wanted to find out what booths vendor 2 was assigned to on or before (less than or equal to) April 20, 2019.

```
SELECT *
 FROM vendor_booth_assignments
 WHERE vendor_id = 3 AND market_date <= "2019-04-20"
 ORDER BY market_date
```

Other Ways to filter

- The filters we have seen so far include numeric, string, and date comparisons to determine if a value in a field is greater than, less than, or equal to a given comparison value.
- Other ways to filter rows based on the values in that row include checking if a field is NULL,
- comparing a string against another partial string value using a wildcard comparison,

- determining if a field value is found within a list of values,
- determining if a field value lies between two other values, among others.

BETWEEN

Last query that we wrote in the previous lecture:

```
SELECT *
  FROM farmers_market.vendor_booth_assignments
  WHERE vendor_id = 2
     AND market_date <= '2019-03-09'
  ORDER BY market_date
```

We can also use the BETWEEN keyword to see if a value, such as a date, is within a specified range of values.

Question: Find the booth assignments for vendor 7 for any market date that occurred between April 3, 2019, and May 16, 2019, including either of the two dates.

```
SELECT *

  FROM farmers_market.vendor_booth_assignments

  WHERE

          vendor_id = 7

        AND market_date BETWEEN '2019-04-03' and '2019-05-16'

  ORDER BY market_date
```

IN - Keyword

Question: Return a list of customers with selected last names - [Diaz, Edwards and Wilson].

Approach 1: we could use a long list of OR comparisons.

```
SELECT
    customer_id,
    customer_first_name,
    customer_last_name
 FROM farmers_market.customer
 WHERE
    customer_last_name = 'Diaz'
    OR customer_last_name = 'Edwards'
    OR customer_last_name = 'Wilson'
```

Approach 2: An alternative way to do the same thing, which may come in handy if you had a long list of names, is to use the IN keyword and provide a comma-separated list of values to compare against.

The IN keyword will return TRUE for any row with a customer_last_name that is in the provided list.

```
SELECT
    customer_id,
    customer_first_name,
```

```
    customer_last_name
FROM farmers_market.customer
WHERE
    customer_last_name IN ('Diaz' , 'Edwards', 'Wilson')
ORDER BY customer_last_name, customer_first_name
```

Both queries have given the same output but whenever you have a long list of names or values that you want to search against, IN is the better and more convenient choice.

Not remembering the spelling of a Name - use case for LIKE

Suppose you're searching for a person at the *customer* table but don't know the spelling of their name.

For example, if someone asked you to look up a customer ID for someone's name (Manuel), but you don't know how to spell the name exactly, you might try searching against a list with multiple spellings, like this:

```
SELECT
    customer_id,
    customer_first_name,
    customer_last_name
FROM customer
WHERE
    customer_first_name IN ("Mann", "Mannuel", "Manuel")
```

Transition → A better way to solve this problem is using LIKE

LIKE keyword

Question: You want to get data about a customer you knew as “Jerry,” but you aren’t sure if he was listed in the database as “Jerry” or “Jeremy” or “Jeremiah.”

All you knew for sure was that the first three letters were “Jer.”

In SQL, instead of listing every variation you can think of, you can search for **partially matched strings** using a **comparison operator called LIKE**, and **wildcard characters**, which serve as a placeholder for unknown characters in a string.

% wildcard

The wildcard character % (percent sign) can serve as a stand-in for any number of characters (including none).

So the comparison LIKE ‘Jer%’ will search for strings that start with “Jer” and have any (or no) additional characters after the “r”:

```
SELECT
    customer_id,
    customer_first_name,
    customer_last_name
 FROM farmers_market.customer
 WHERE
    customer_first_name LIKE 'Jer%'
```

Here are some examples showing different `LIKE` operators with '%' and '_' wildcards:

LIKE Operator	Description
WHERE CustomerName LIKE 'a%'	Finds any values that starts with "a"
WHERE CustomerName LIKE '%a'	Finds any values that ends with "a"
WHERE CustomerName LIKE '%or%'	Finds any values that have "or" in any position
WHERE CustomerName LIKE '_r%'	Finds any values that have "r" in the second position
WHERE CustomerName LIKE 'a__%'	Finds any values that starts with "a" and are at least 3 characters in length
WHERE ContactName LIKE 'a%o'	Finds any values that starts with "a" and ends with "o"

IS NULL

You have learnt how to find and treat missing values in Python but what happens when your **database contains missing values.**

- In the product table, the product_size field is optional, so it's possible to add a record for a product with no size.

Question: Find all of the products from the product table without sizes.

```
SELECT *
    FROM farmers_market.product
    WHERE product_size IS NULL
```

Blank vs NULL

Note: Keep in mind that "blank" and NULL are not the same thing in database terms.

- If someone asked you to find all products that didn't have product sizes, you might also want to check for blank strings, which would equal '', or rows where someone entered a space or any number of spaces into that field.
- The TRIM() function removes excess spaces from the beginning or end of a string value, so if you use a combination of the TRIM() function and blank string comparison, you can find any row that is blank or contains only spaces.
- In this case, the "Red Potatoes - Small" row, shown in Figure 3.14, has a product_size with one space in it, ' ', so could be found using the following query:

```
SELECT *
FROM farmers_market.product
WHERE
    product_size IS NULL
    OR TRIM(product_size) = ""
```

Warning about NULL values

- You might wonder why the comparison operator IS NULL is used instead of equals NULL just like numbers.
- **NULL is not actually a value, it's the absence of a value, so it can't be compared to any existing value.**
- If your query were filtered to ***WHERE product_size = NULL***, no rows would be returned, even though there is a record with a NULL product_size, **because nothing "equals" NULL, even NULL.**

- If you wanted to return all records that don't have NULL values in a field, you could use the condition "[field name] **IS NOT NULL**" in the WHERE clause.

Filtering Using Subqueries

When the IN list comparison was demonstrated earlier, it used a hard-coded list of values.

- What if you wanted to filter to a list of values that was returned by another query?

- In other words, you wanted a dynamic list. There is a way to do that in SQL, using a subquery (a query inside a query).

Question: Analyze purchases made at the farmer's market on days when it rained.

There is a value in the *market_date_info* **table** called *market_rain_flag* that has a value of **0 if it didn't rain** while the market was open and **a value of 1 if it did**.

- 0 - it didn't rain
- 1 - it did

First, let's write a query that gets a list of market dates when it rained, using this query:

```
SELECT market_date, market_rain_flag
  FROM farmers_market.market_date_info
```

```
 WHERE market_rain_flag = 1
```

- Now let's use the list of dates generated by that query to return purchases made on those dates.
- Note that when using a query in an IN comparison, you can **only return the field you're comparing to**, so we will not include the *market_rain_flag* field in the following subquery.
- Therefore, the query inside the parentheses just returns the dates.
- The "outer" query looks for *customer_purchases* records with a market_date value in that list of dates.

```
 SELECT
    market_date,
    customer_id,
    vendor_id,
    quantity * cost_to_customer_per_qty price
  FROM farmers_market.customer_purchases
  WHERE
    market_date IN
        (
        SELECT market_date
        FROM farmers_market.market_date_info
        WHERE market_rain_flag = 1
        )
LIMIT 5
```

So, WHERE works when you have conditional statements to filter out your rows.

But what if, instead of using conditional statements to filter rows, you want a column or value in your dataset to be based on a conditional statement?

For example, instead of filtering your results for transactions over $50, say you just want to return all rows and create a new column that flags each transaction as being above or below $50?

This is where CASE statements come in.

CASE Statements

Note: If you're familiar with scripting languages like Python that use "if " statements, you'll find that SQL handles conditional logic somewhat similarly, with different syntax.

Conditional flow: "If [one condition] is true, then [take this action].
Otherwise, [take this other action]."

For example: "If the weather forecast predicts it will rain today, then I'll take an umbrella with me. Otherwise, I'll leave the umbrella at home."

In SQL, the code to describe this type of logic is called a CASE statement, which uses the following syntax:

```
CASE
        WHEN [first conditional statement]
          THEN [value or calculation]
        WHEN [second conditional statement]
          THEN [value or calculation]
        ELSE [value or calculation]
END
```

This statement indicates that you want a column to contain different values under different conditions.

If we put the umbrella example into this form:

```
CASE
```

```
        WHEN weather_forecast = 'rain'
            THEN 'take umbrella'
        ELSE 'leave umbrella at home'
END
```

Execution Method

- The WHENs are evaluated in order, from top to bottom, and the first time a condition evaluates to TRUE, the corresponding THEN part of the statement is executed, and no other WHEN conditions are evaluated.

Question: Find out which vendors primarily sell fresh produce and which don't.

The vendors we want to label as "Fresh Produce" have the word "Fresh" in the *vendor_type* column.

We can use a CASE statement and the LIKE operator to create a new column, which we'll alias ***vendor_type_condensed***, that condenses the vendor types to just "Fresh Produce" or "Other":

```
SELECT
    vendor_id,
    vendor_name,
    vendor_type,
    CASE
        WHEN LOWER(vendor_type) LIKE '%fresh%'
          THEN 'Fresh Produce'
        ELSE 'Other'
    END AS vendor_type_condensed
 FROM farmers_market.vendor
```

We're using the LOWER() function to lowercase the vendor type string, because we don't want the comparison to fail because of capitalization.

Creating binary flags using CASE

The Farmer's Markets in our database all occur on Wednesday evenings or Saturday mornings.

Many machine learning algorithms won't know what to do with the words "Wednesday" and "Saturday" that appear in our database:

Q: Add a column that identifies whether that day is weekend or not.

```
SELECT
    market_date,
    market_day
 FROM farmers_market.market_date_info
 LIMIT 5
```

Binary Encoding: But, the algorithm could use a numeric value as an input.

So, how might we turn this string column into a number?

One approach we can take to including the market day in our dataset is to generate a binary flag field that indicates whether it's a weekday or weekend market.

We can do this with a CASE statement, making a new column that contains a 1 if the market occurs on a Saturday or Sunday, and a 0 if it doesn't,
We'll call this field "*weekend_flag*,"

```
SELECT
    Market_date,
        CASE
                WHEN market_day = 'Saturday' OR market_day = 'Sunday'
                THEN 1 ELSE 0
        END AS weekend_flag
 FROM farmers_market.market_date_info
 LIMIT 5
```

Grouping or Binning Continuous Values Using CASE

We had a query that filtered to only customer purchases where an item or quantity of an item cost over $50, by putting a conditional statement in the WHERE clause.

But let's say we wanted to return all rows, and instead of using that value as a filter, only **indicate whether the cost was over $50 or not**.

```
SELECT
    market_date,
    customer_id,
    vendor_id,
    ROUND(quantity * cost_to_customer_per_qty, 2) AS price,
    CASE
            WHEN quantity * cost_to_customer_per_qty > 50
            THEN 1 ELSE 0
    END AS price_over_50
 FROM farmers_market.customer_purchases
LIMIT 10
```

Question: Put the total cost to customer purchases into bins of

- under $5.00,
- $5.00–$9.99,
- $10.00–$19.99, or
- $20.00 and over.

We could accomplish that with a CASE statement in which we surround the values after the THENs in single quotes to generate a column that contains a string label,

```
SELECT
        market_date,
```

```
        customer_id,
        vendor_id,
        ROUND(quantity * cost_to_customer_per_qty, 2) AS price,
        CASE
            WHEN quantity * cost_to_customer_per_qty < 5.00
                THEN 'Under $5'
            WHEN quantity * cost_to_customer_per_qty < 10.00
                THEN '$5-$9.99'
            WHEN quantity * cost_to_customer_per_qty < 20.00
                THEN '$10-$19.99'
            WHEN quantity * cost_to_customer_per_qty >= 20.00
                THEN '$20 and Up'
            END AS price_bin
    FROM farmers_market.customer_purchases
    LIMIT 10
```

Can also be used for categorical encoding.

```
SELECT
    booth_number,
    booth_price_level,
    CASE
        WHEN booth_price_level = 'A' THEN 1
        WHEN booth_price_level = 'B' THEN 2
        WHEN booth_price_level = 'C' THEN 3
    END AS booth_price_level_numeric
 FROM farmers_market.booth
 LIMIT 5
```

SUMMARY

In this lecture, you learned

- How to filter rows using keywords such as LIKE, IN,
- How to filter out rows with NULL values using IS NULL
- How to use Subqueries.
- SQL CASE statement syntax for creating new columns with values based on conditions.
- You also learned how to consolidate categorical values into fewer categories, create binary flags, bin continuous values, and encode categorical values.

You should now be able to describe what the following two queries do.